Lecture Note 9: Overview of File Systems and I/O Systems

CSS225 Operating Systems

School of ICT, SIIT, Thammasat University

Overview of File Systems

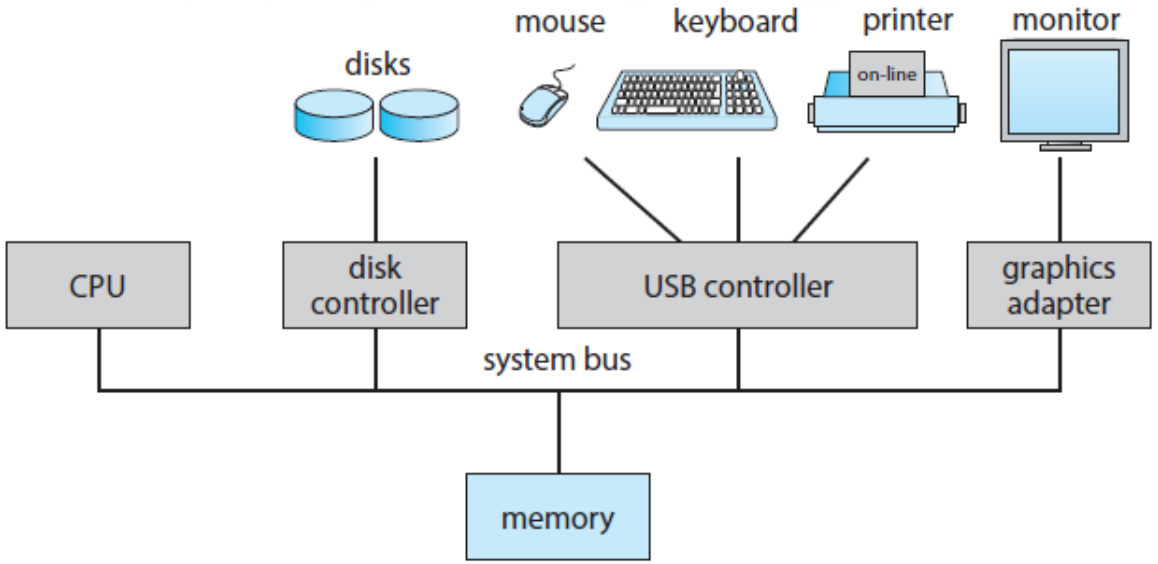
1) File-system management is a service provided by the operating system.
2) A file system is about the space management in nonvolatile storage devices (such as the hard disk).
   - The file system provides efficient and convenient access to the storage device by allowing data to be stored, located, and retrieved easily.
3) A file system poses two quite different design problems.
   a) The first problem is defining how the file system should look to the user. This task involves defining:
      i. a file and its attributes,
      ii. the operations allowed on a file,
      iii. the directory structure for organizing files.
   b) The second problem is creating algorithms and data structures to map the logical file system (data) onto the physical secondary-storage devices.

Overview of File Systems

4) A file system consists of two distinct parts:
   - Files: Each file stores related data. It is the smallest logical storage unit.
   - Directories/Folders: A directory structure provides and organizes information about all the files in the system.

Overview of I/O Systems

1) The two main jobs of a computer system are processing and input/output (I/O).
2) A role of the operating system in computer systems is to manage and control I/O operations and I/O devices.
   - The control of devices connected to the computer is a major concern of operating-system designers.
3) I/O-device technology exhibits two conflicting trends:
   - We see an increasingly broad variety of I/O devices. We have many new devices.
   - On the one hand, we want to set standardization of software and hardware interfaces to these devices (i.e., want the to use the same commands/operations/interfaces).



Outlines

1. File Concept
2. Directory Structure
3. Protection
4. File-System Implementation
5. Input and Output (I/O) System

1. File Concept

File: A Sequence of Blocks of Data

1) A file is a collection of data (from the user's point of view), which is equivalent to a collection of bytes (computer system's point of view).
   - A file consists of a series of bits 0 and 1:

     …101011101101010001010110111010010 1…

2) A simple structure of a file is that it is divided into a sequence of data blocks/records, where each data block consists of a number of bytes (for example, 512 Bytes).

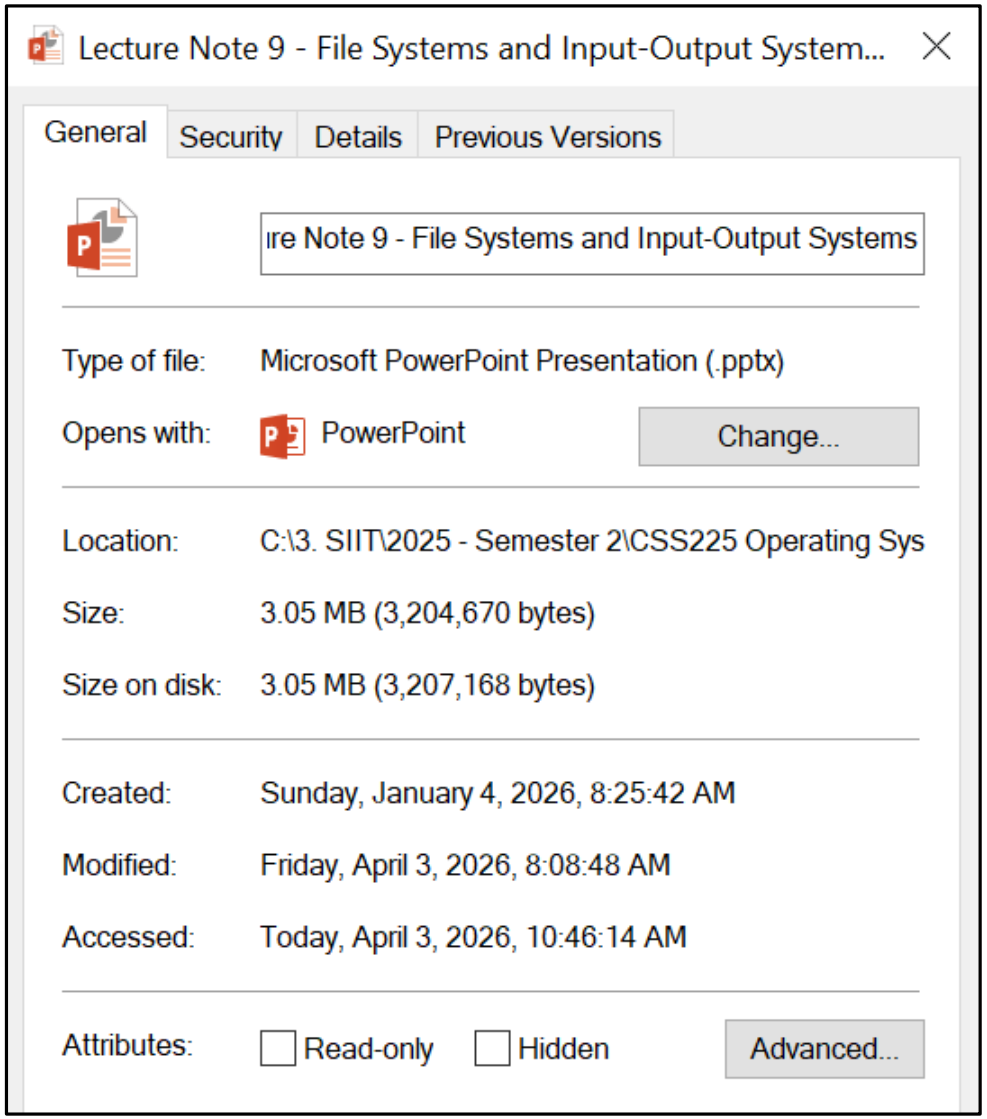
1001…0111	1111…0011	0101…0100	…	1011…1001
Data Block 0	Data Block 1	Data Block 2		Data Block N-1

Facts about Files

- A file represents a collection of data stored in a physical storage device (e.x., hard drive).
  - A file is an abstraction entity and a logical view provided by the operating system.
  - A file is the smallest logical storage unit.
- A role of the operating system on files:
  - The operating system abstracts from the physical properties of its storage devices to define a logical storage unit called a file (file system's user interface).
  - Files are mapped by the operating system onto physical devices (file system's implementation). Specifically, a computer system can read/write the files stored in physical devices.

Information about a File: File Attributes or Metadata

- The operating system provides information about a file via file attributes or metadata.
- The file attributes vary from one operating system to another.
- Basically, they consist of
  - Name, type, location, size, date, protection, etc.



File Types/Structures

- The operating system and application programs should recognize and promptly open files. To do so, we add a file type (file's last name/extension) to each file.
  - Based on the file type, the file will be opened by a suitable (default) program when being double clicked.
  - A file type can be added to a file either by a user or a program who creates it.
  - For example:
    - Files with the *docx* extension are opened by the MS-Word program.
    - Files with the *pdf* extension can be opened by the Adobe Reader.

File Types/Structures

- A program can read and open a file because the file structure matches the program's expectation.
  - Generally, a file is identified by its file type.
- The operating system must be able to read and execute at least one file type, which is the executable file (.exe).
  - The operating system know the file structure of the executable file (.exe).
    - For example, the OS knows where the first instruction to read.

File Operations

The operating system provides system calls such that a process can do the following operations to a file.

1) Creating a file.
2) Writing a file.
3) Reading a file.
4) Repositioning within a file (move the pointer/cursor in a file).
5) Deleting a file.
6) Truncated a file (= erase the contents in the file but keep the file's attributes).

Open-File Tables

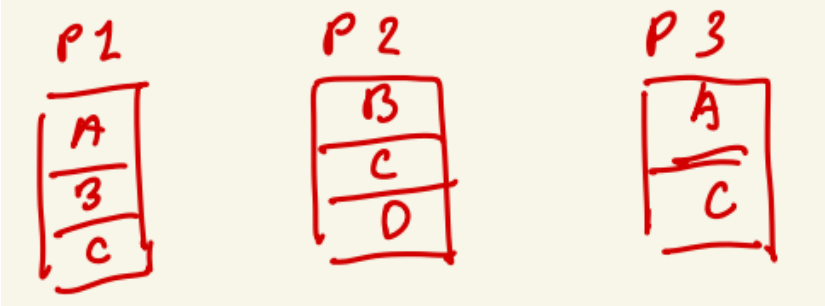
Most of the file operations are involved opening files. The operating system uses 2 tables to keep tracks of these open files.

- The per-process open-file table stores information of the files that are open by a process.
- The system-wide open-file table stores the open-file entry and the open count (i.e., the number of processes that are opening this file).

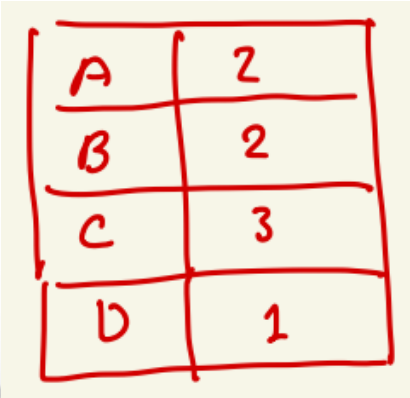
Example: Open-File Tables

Process P1 is opening files A, B, C. Process P2 is opening files B, C, D. Process P3 is opening files A, C.

1) Therefore, we have the following "per-process Open File Tables":



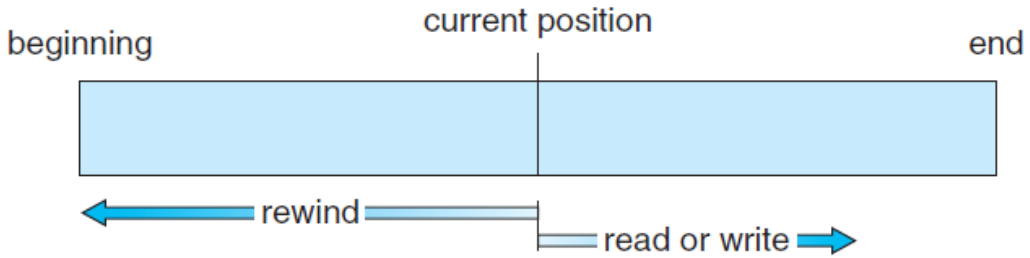
2) Therefore, we have the this "system-wide Open File Tables":



Access Methods (Read/Write) for Files

There are two basic methods on read/write data into a file.

1) Sequential Access: Information in the file is processed in order, one record after the other. Sequential access, which is depicted below, is based on a tape model of a file.



Access Methods (Read/Write) for Files

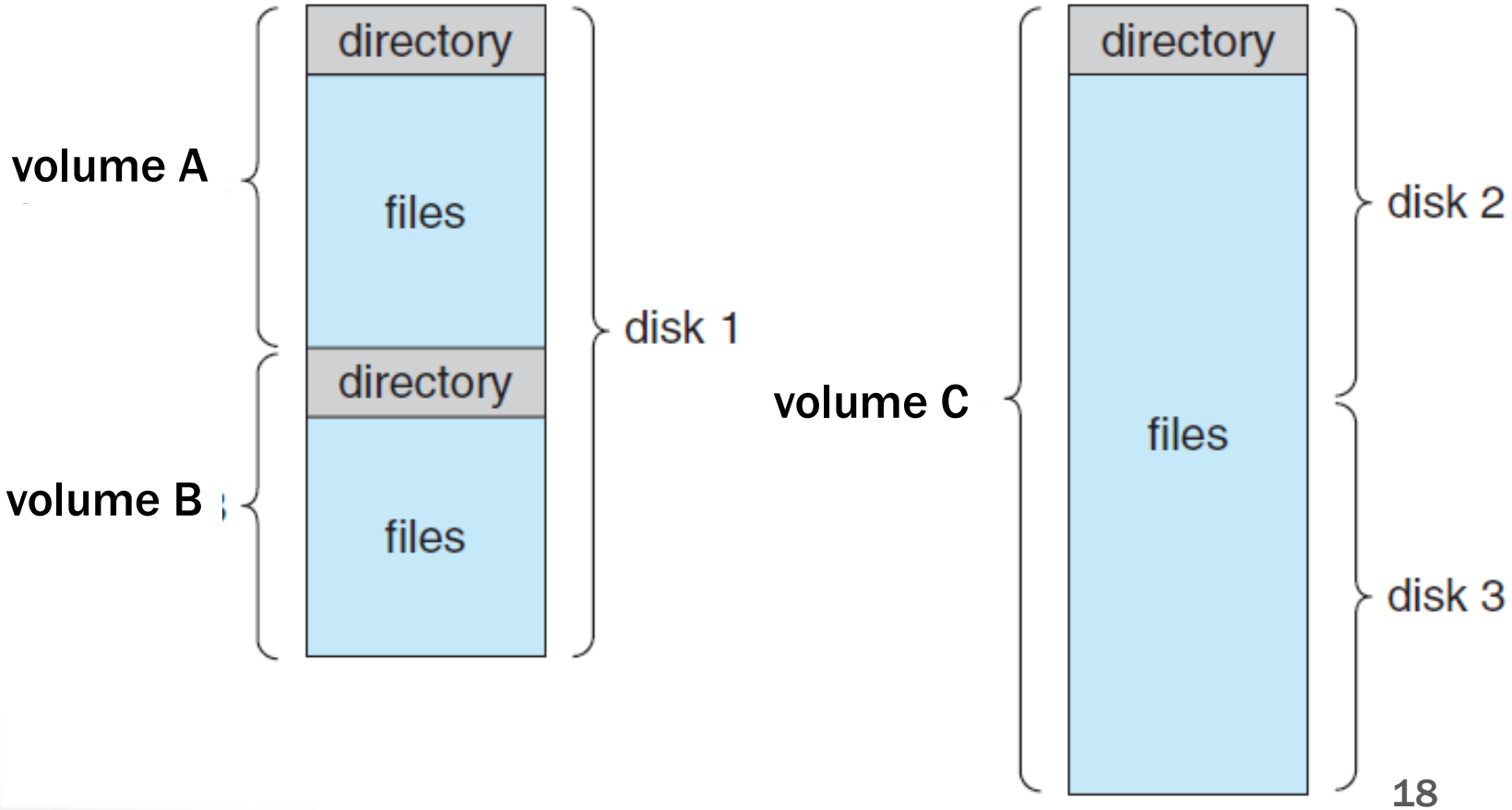
2) Direct Access:

- A file is made up of fixed-length logical records (blocks of bytes) that allow programs to read and write records rapidly in no particular order.
- The direct-access method is based on a disk model of a file, since disks allow random access to any file block.
- For direct access, the file is viewed as a numbered sequence of blocks or records. Thus, we may read block 14, then read block 53, and then write block 7. There are no restrictions on the order of reading or writing for a direct-access file.

2. Directory Structures

Partition

- A storage device (e.x., a disk) can be partitioned into partitions (or quarters).
  - Partitioning is useful for limiting the sizes of individual file systems.
- Each partition (or quarter) will be installed a file system.
  - Directory
  - Files



Volume

- The partition with a file system is called a volume (in computer).
  - A volume can be thought of as a virtual disk.
  - A volume consists of files and a volume table of contents (also known as a device directory or directory)

- The difference between a partition and volume.
  - A partition is a space crafted out of a disk.
  - A volume is a partition that has been formatted into a filesystem.

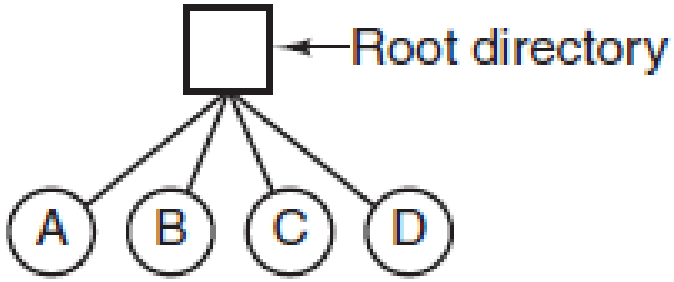
Directory Structures

1) Since there are a lot of files (thousands, even millions) in the computer system, we organize these files into groups (called directories) such that they are manageable.
2) Common types of directory structures:
   a) Single-level directory
   b) Two-level directory
   c) Tree-structured directory
   d) Acyclic-graph directory
   e) General graph directory

Notations:
- A Directory
- A File

a) Single-Level Directory (Only One Directory)

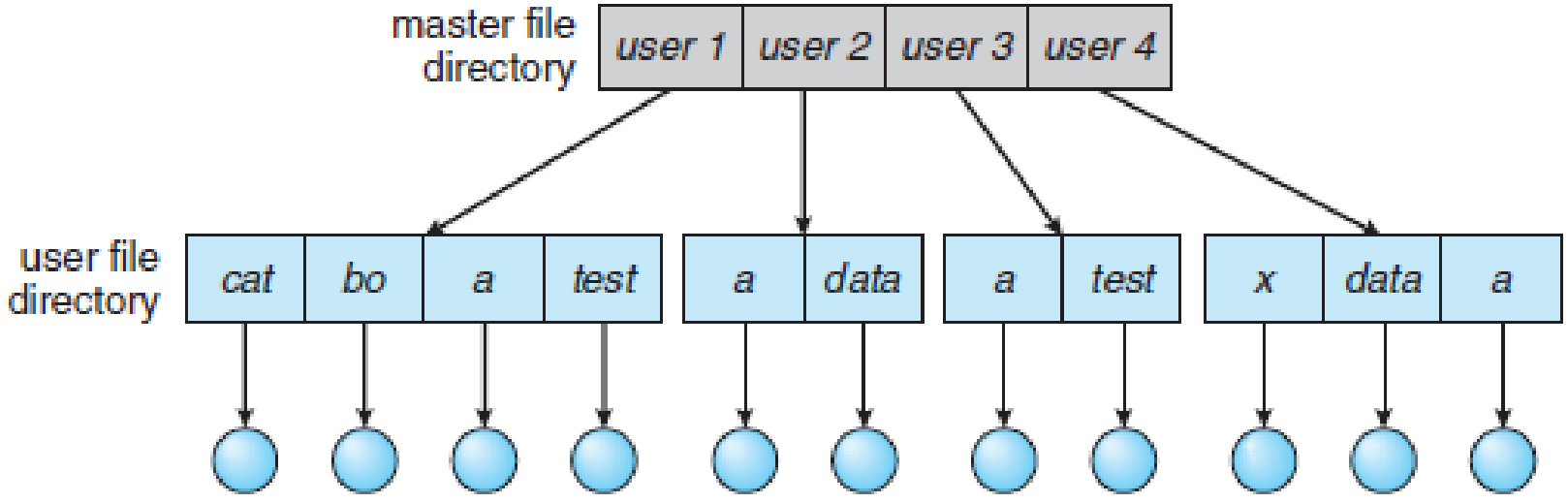
- The simplest form of directory system is having one directory containing all the files.
- This directory is called the root directory.
- All users of this computer system share this directory.



- Advantage: It is the simplest directory structure.
- Disadvantage:
  - Naming problem. Since two or more files cannot have the same name in the same directory, the users may find a difficulty to set up file names.
  - Searching problem. A lot of files in one directory.

b) Two-Level Directory

- The two-level directory is an extension of the single-level directory such that each user has his/her own directory (user directory).
  - A user directory is divided further into directories. Each directory consists of files.
- The two-level directory structure consists of
  - *Master file directory* (MFD) which is separated into users.
  - *User file directory* (UFD) which consists of a set of directories according to each user.

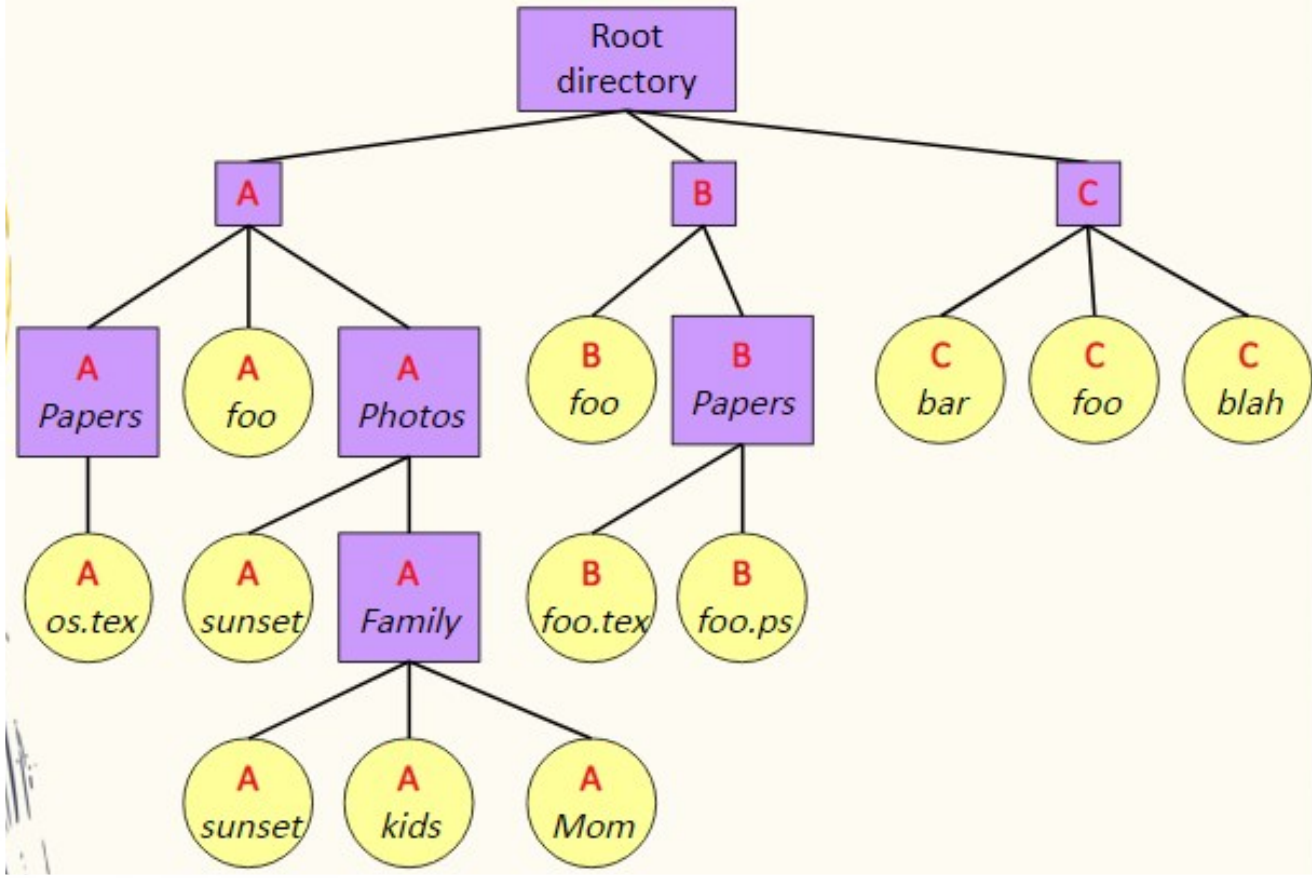


b) Two-Level Directory

- A two-level directory can be thought of as a tree, or an inverted tree, of height 2. The root of the tree is the MFD while the leaves are files.
- Advantage: Solve the name problem among users and improve searching efficiency.
- Disadvantage: This model isolates users from each other. No one can access the other users' files.

c) Tree-Structure Directory

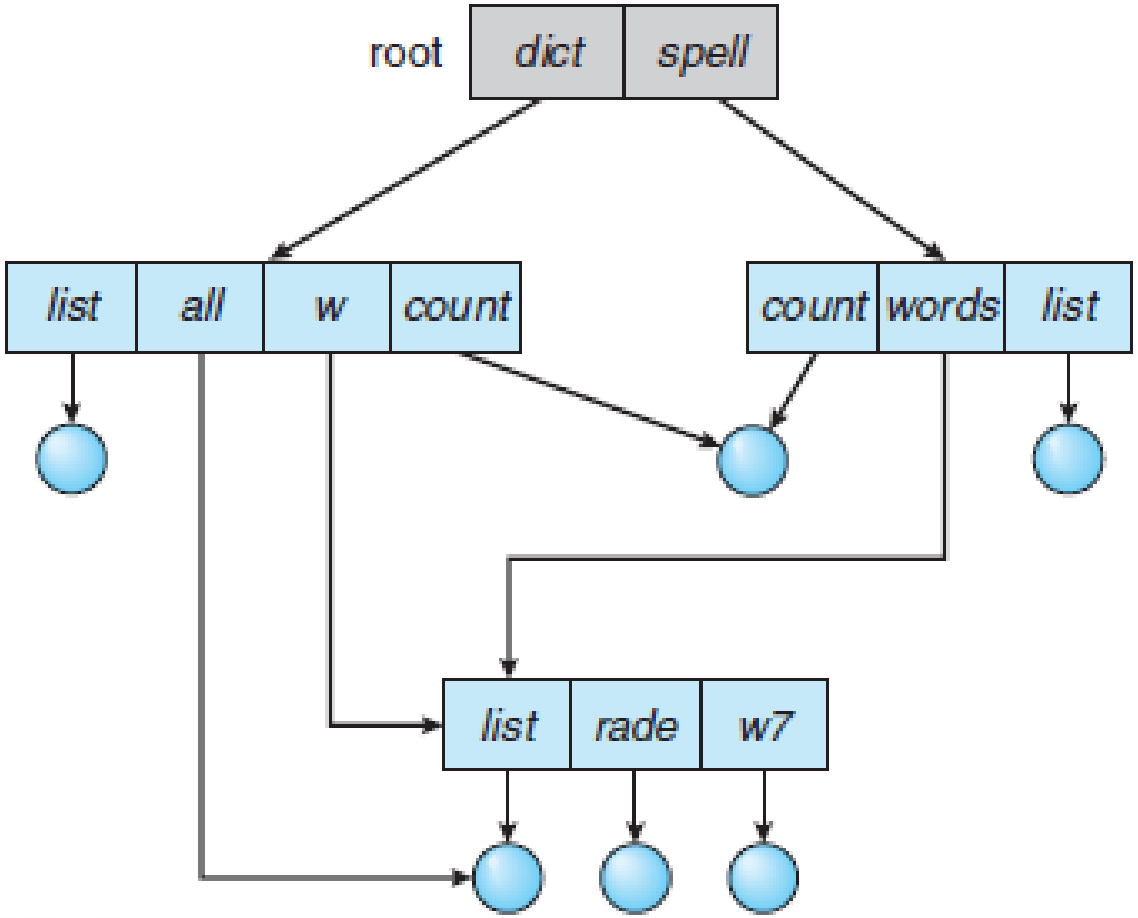
- It is a directory structure with arbitrary levels. It is a commonly used structure.
- Key characters of this structure:
  - There are (sub)directory for each user.
  - It allows a user to create (sub)directories freely.
  - Each file has a unique path name. Two files can have the same name if they are located at different path names.
  - A user can access (sub)directories and files of another user.



d) Acyclic-Graph Directory

The key character of this structure is to allow to have shared directories or files.

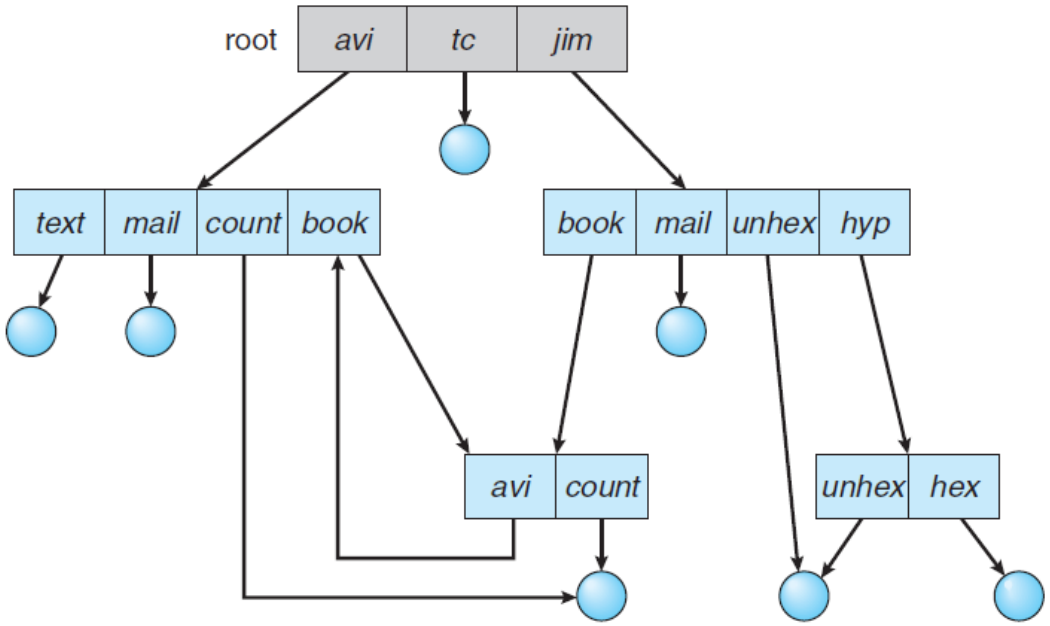
- Shared directories or files mean that these directories or files will appear in other locations and be accessed via those locations.
- As a result, these shared directories or files are in the file system in two or more places at once.
- Only one shared file/directory exists. Any changes on this file/directory will be immediately visible by the others.
- The shared directories/files are not equivalent to copying of the original directories/files.
- There will be more than one path name to the shared file/directory.



e) General-Graph Directory

It is a tree structure that allows a cycle occurring in the directory structure.

- For example, the directory "book" contains a directory "avi" and the directory "avi" contains the directory "book".
- A problem will happen, for example, when we are searching inside one of these directories. We will encounter an infinite loop.
- As a solution, for the above problem, we can limit the number of directories that will be accessed during a search.



Path Name

A path name is used to identify the location of a file in a computer.

- For example, if we would like to identify the path name of the file os.tex which is stored in the directory Papers while the directory Papers is inside the directory A (which is under the root (Unix) or C (Windows)):
  1) Unix path name has the following pattern (note that the first "/" is the root directory):
     - /A/Papers/os.tex
  2) Windows path name must include the volume where the file stored:
     - C:\A\Papers\os.tex
- Note that the Unix uses "/" while Windows uses "\" to specify a series of directories and the file.

Path Name

Types of path names:

- For example, consider the following Unix path name of the file os,tex:

/A/Papers/os.tex

1) An absolute path name begins at the root and follows a path down to the specified file.
   - For example, /A/Papers/os.tex
2) A relative path name defines a path to the file from the current directory. For example:
   - If the current directory is /A
   - The relative path name is /Papers/os.tex

3. Protection

Scenario: Consider a shared computer system where there might be many users sharing a computer system. How could we protect our files and directories?

We can set permissions on:

- who can access our files or directories.
- what they can do on our files or directories.

File and Directory Access Permissions

The owner of a file or directory can define specific Access Permissions to control how other users interact with this file or directory:

- Read (r): Allows a user to view or read the contents of the file.
- Write (w): Allows a user to modify, update, or overwrite the file.
- Execute (x): Allows the system to load the file into memory and run it as a program or script.

User Categories

- Many operating system (ex., Linux) classifies users into three types/categories. Assume that we are setting a permission to access a file/directory.
  - Owner. The user who created this file/directory is the owner.
  - Group. A set of users who are in the same group of the owner.
  - Others or Universe. The other users who are not in the same group of the owner.
- The owner of a file is able to set permissions (for other users) to access this file.
  - For example, in Linux, we will use the command ***chmod*** to set permissions on a file.

Example: User Categories

Assume that we have the following users: A, B, C, D, E. They are divided/assigned into the following user groups:

- User Group 1: A, B.
- User Group 2: C.
- User Group 3: D, E.

Assumes that the user A created a file “os.doc”. According to the user category to access this file above, we will have:

- Owner: A.
- Group: B.
- Others/Universe: C, D, E.

Example: Permissions

- In Unix/Linux, the file/directory owner can set permissions on who can read (r), write (w), or execute (x) on his/her file/directory.
- When you are at the shell prompt, you type the command ***ls –l*** to see a list of directories and files in this directory. You then can see the permissions represented by the first 10 alphabets on each file/directory:

	Owner			Group			Others		
d/f	r	w	x	r	w	x	r	w	x

```
seksan@ubuntu:~$ ls -l
total 48
drwxr-xr-x 2 seksan seksan 4096 Dec 24 10:23 Desktop
drwxr-xr-x 5 seksan seksan 4096 Jan 11 11:51 Documents
drwxr-xr-x 2 seksan seksan 4096 Dec 24 10:23 Downloads
-rw-r--r-- 1 seksan seksan 8980 Dec 24 10:16 examples.desktop
```

Example: Permissions

Consider a permission set up on a file “os.doc” as following (excluding the first letter to indicate either a file or a directory):

rwxrw-r--

1) What are the actions that the owner can do with this file?

2) What are the actions that the users in Group can do with this file?

3) What are the actions that the users in Universe can do with this file?

Example: Permissions

Assume that we have the following users: A, B, C, D, E. They are divided/assigned into the following user groups:

- User Group 1: A, B.
- User Group 2: C.
- User Group 3: D, E.

Assumes that the user A created a file “os.doc”. According to the access control set up in the previous page, we will have:

- User A can read, write, execute.
- User B can read, write.
- User C, D, E can read.

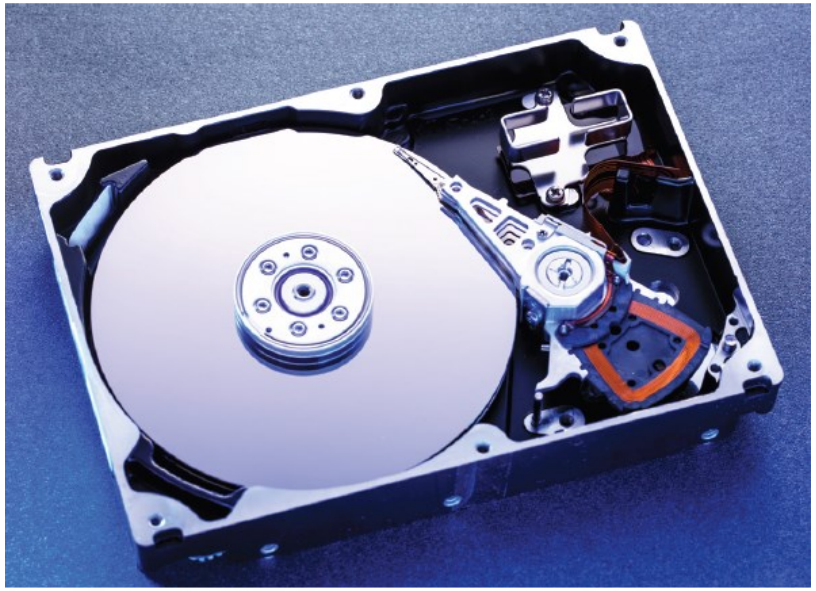
4. File-System Implementation

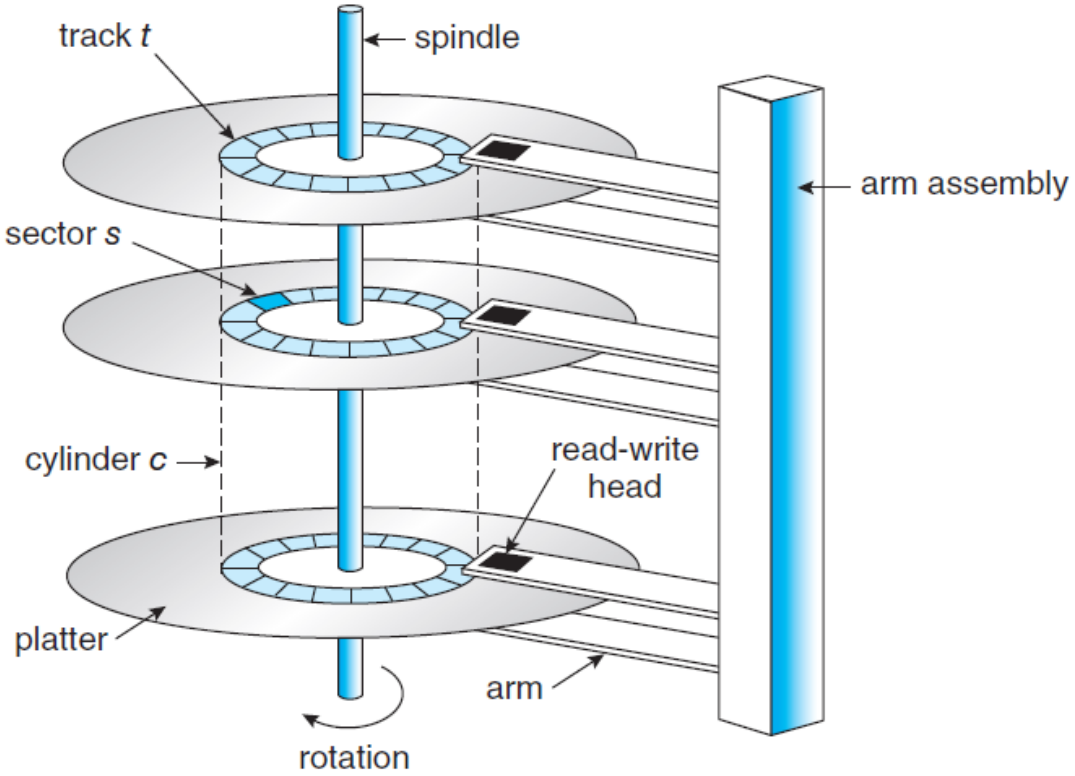
Now it is time to turn from the user's view of the file system to the implementor's view.

- Implementors are interested in how files and directories are stored, how disk space is managed, and how to make everything work efficiently and reliably.

Hard Disk (Magnetic Disk)

A simple structure of a hard disk is shown below. It consists of several disk platters. The two surfaces of a platter are covered with a magnetic material. We store information by recording it magnetically on the platters, and we read information by detecting the magnetic pattern on the platters.



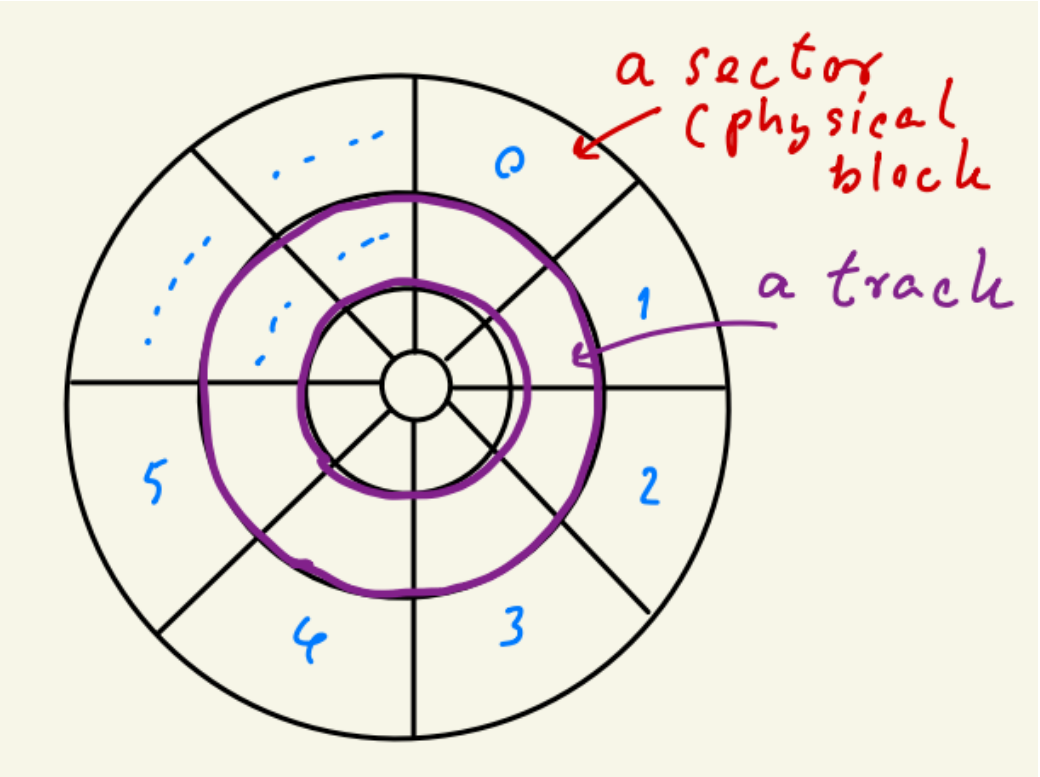


Hard Disk (Magnetic Disk)

- A read–write head flies just above each surface of every platter.
- The heads are attached to a disk arm that moves all the heads as a unit.
- The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- The set of tracks at a given arm position make up a cylinder. There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors.
- Each sector has a fixed size and is the smallest unit of transfer. The sector size was commonly 512 bytes until around 2010. At that point, many manufacturers start migrating to 4KB sectors.

Layout of a Disk Platter

1) The disk platter is divided into tracks.
2) Each track is divided into sectors (i.e., physical blocks, disk blocks).
3) All sectors have the same size.
4) Each sector can be identified by a sector number (e.x., 0, 1, 2, …).
5) A data block of a file will be stored in a free sector.

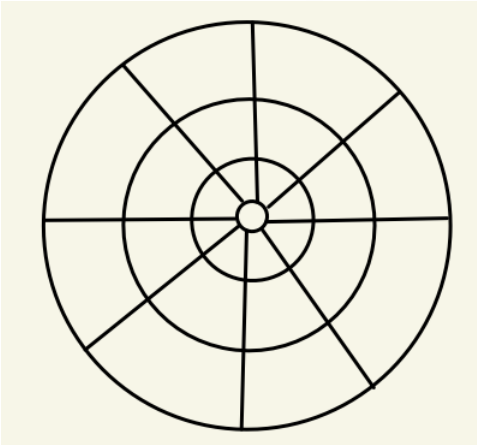


Implementing Files: Scenario

1) A file is divided into data blocks.

1001…0111	1111…0011	0101…0100	…	1011…1001
Data Block 0	Data Block 1	Data Block 2		Data Block N-1

2) A disk platter is divided into equal sectors (= physical blocks, disk blocks).



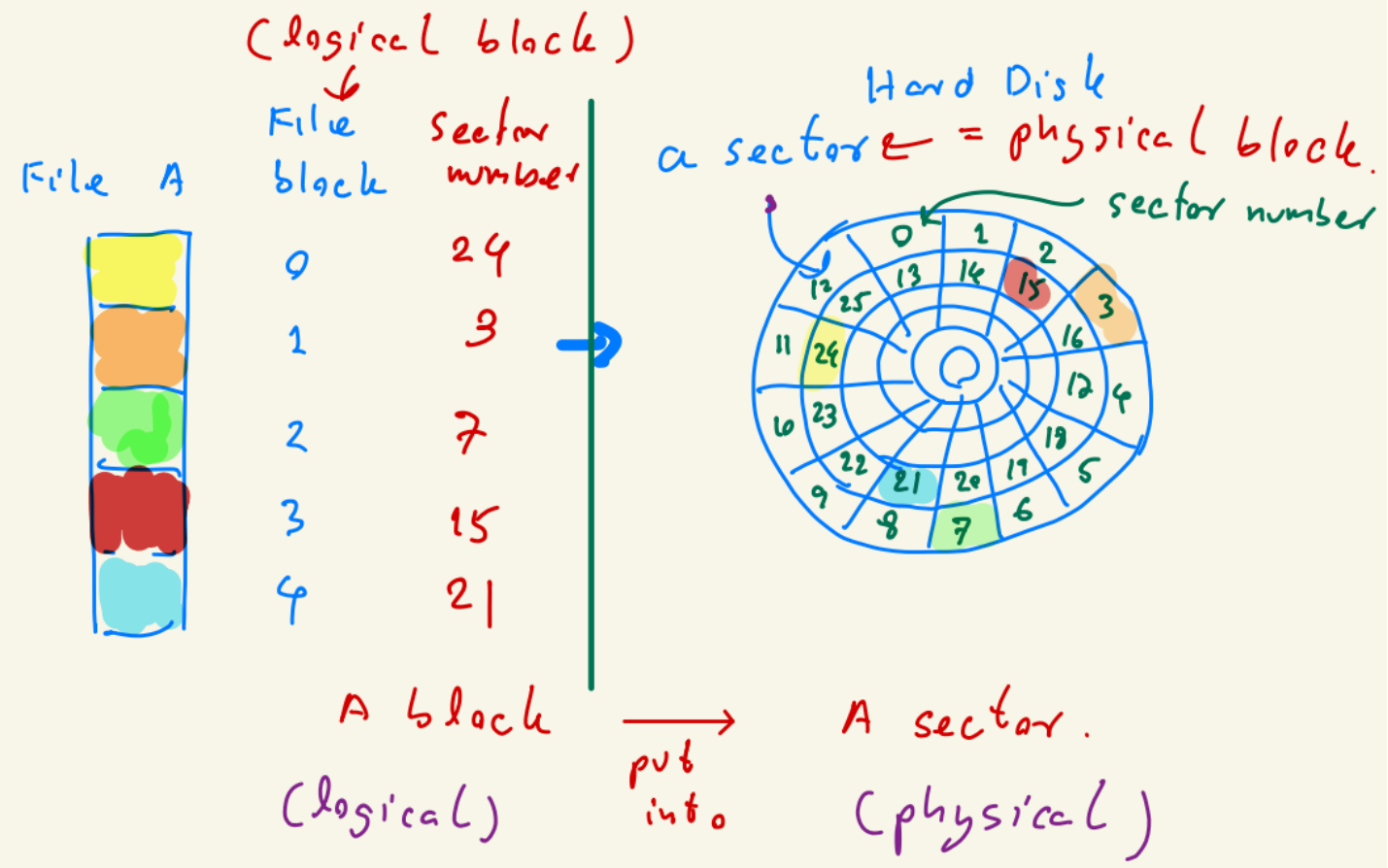
3) A data block size is equal to the sector sizes.

4) Implementing a file into a hard disk is related to where/how to put the data blocks of this file into sectors of the hard disk and keep track on them.

Data Block	Sector Number
0	50
1	14
2	26

Implementing Files: Scenario

4) Implementing a file into a hard disk is related to where/how to put the data blocks of this file into sectors of the hard disk and keep track on them.



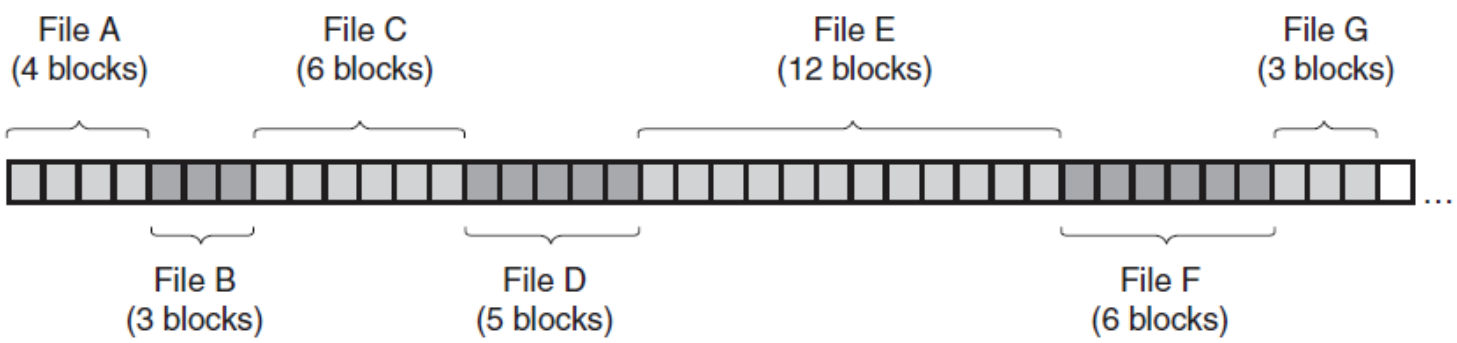
Implementing Files: Approaches

There are 4 approaches:

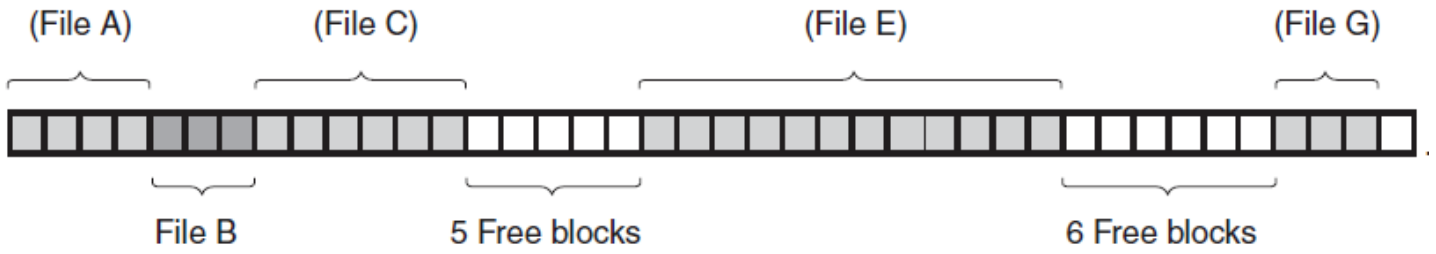
1) Contiguous allocation
2) Linked-List Allocation
3) Linked-List Allocation Using a Table in Memory
4) I-nodes

Implementing Files

1) **Contiguous Allocation**. We allocate the data block of each file next to each other (cannot separate them) in the sectors in the hard disk as shown in the figure below. For example, 4 data blocks of the File A are put next to each other (similar to B, C, D, E, F, and G).

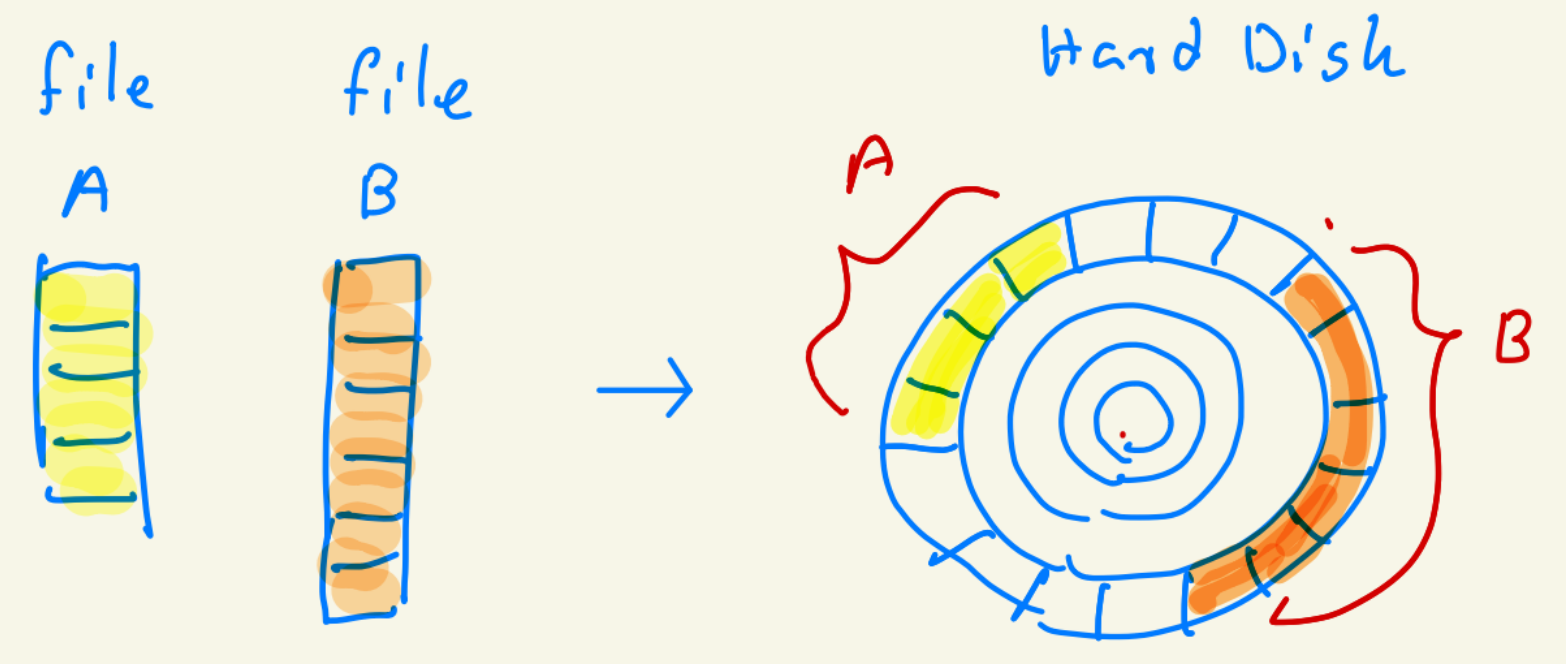


Drawback: Since some files might be removed, the disk will become fragmented (holes) as shown below (D and F are deleted).



Example: Contiguous Allocation

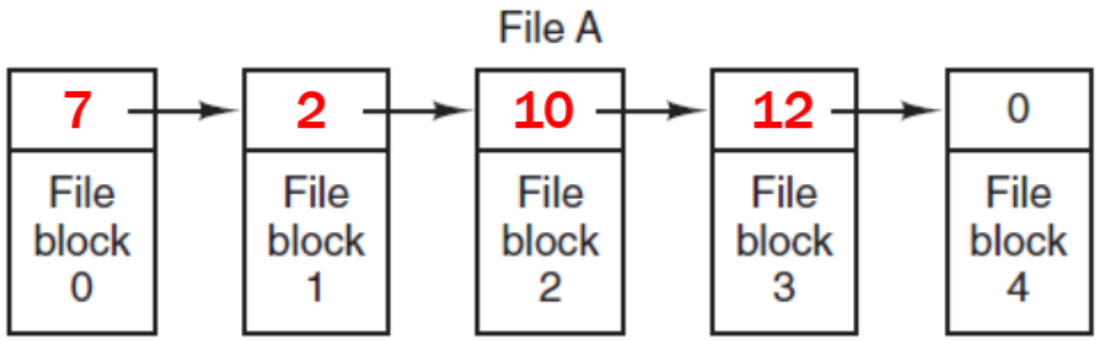
- All data blocks for File A must be stored in contiguous sectors on the disk.
- All data blocks for File B must be stored in contiguous sectors on the disk.



Implementing Files

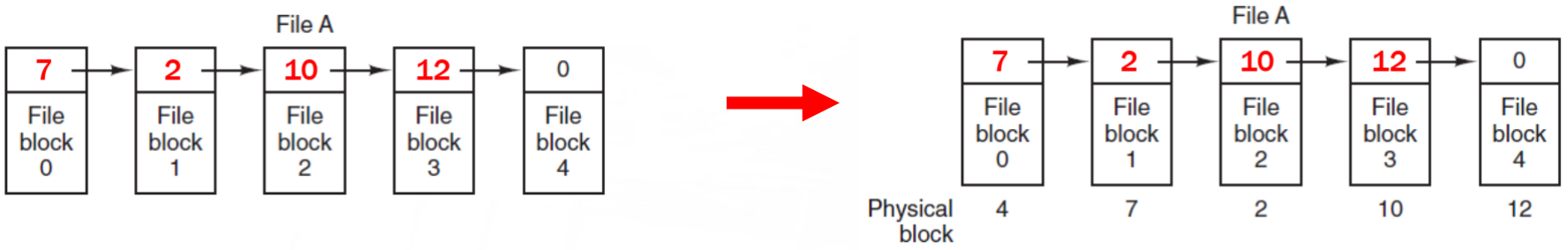
2) **Linked-List Allocation**. The second method for storing files is to keep each one as a linked list of disk blocks, as shown in the figure below.

- Unlike contiguous allocation, the data blocks of a file can be stored in any sectors (no need to be next to each other).
- The first word of each data block is used as a pointer to the next sector.
- The remaining words of the block is for data.
- The sector who stores the data block 0 will be given in the directory.

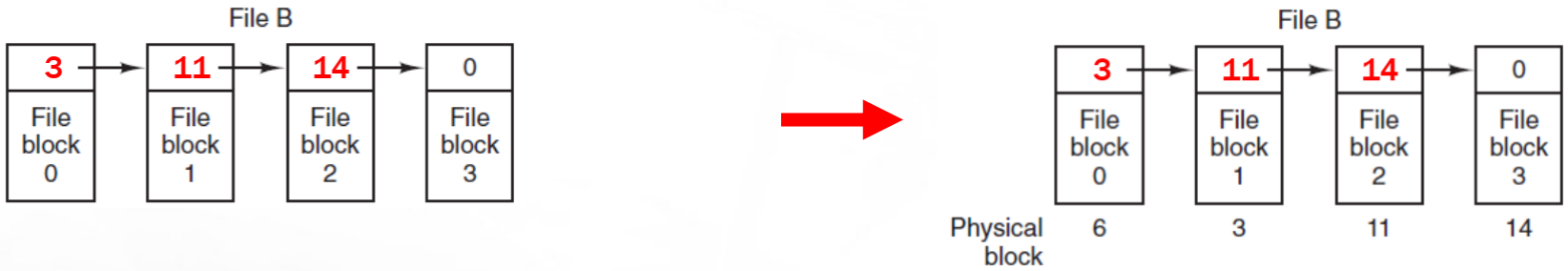


Example: Linked-List Allocation

1) File A consists of 5 data blocks. The data block 0 is allocated at the sector 4 (specified in the directory). By looking at the first word of each data block, we know that the data blocks of File A is stored in the following sectors (in order): 4, 7, 2, 10, 12.



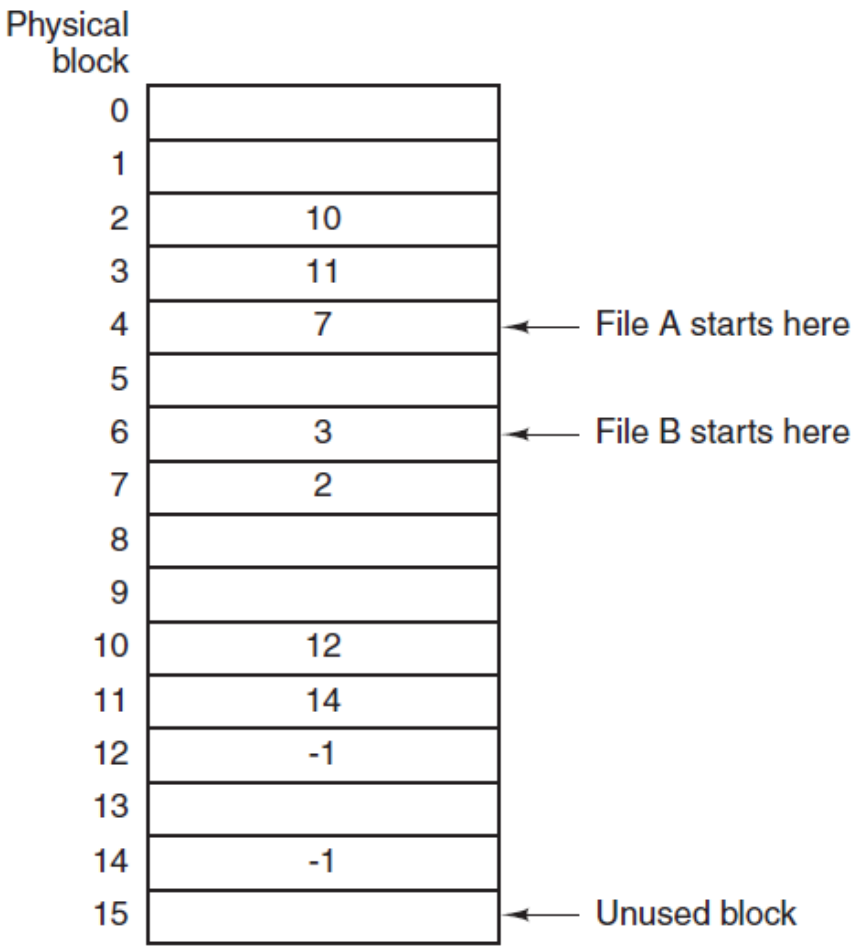
2) File B consists of 4 data blocks. The data block 0 is allocated at the sector 6 (specified in the directory). By looking at the first word of each data block, we know that the data blocks of File B is stored in the following sectors (in order): 6, 3, 11, 14.



Implementing Files

3) **Linked-List Allocation using a Table in Memory**.

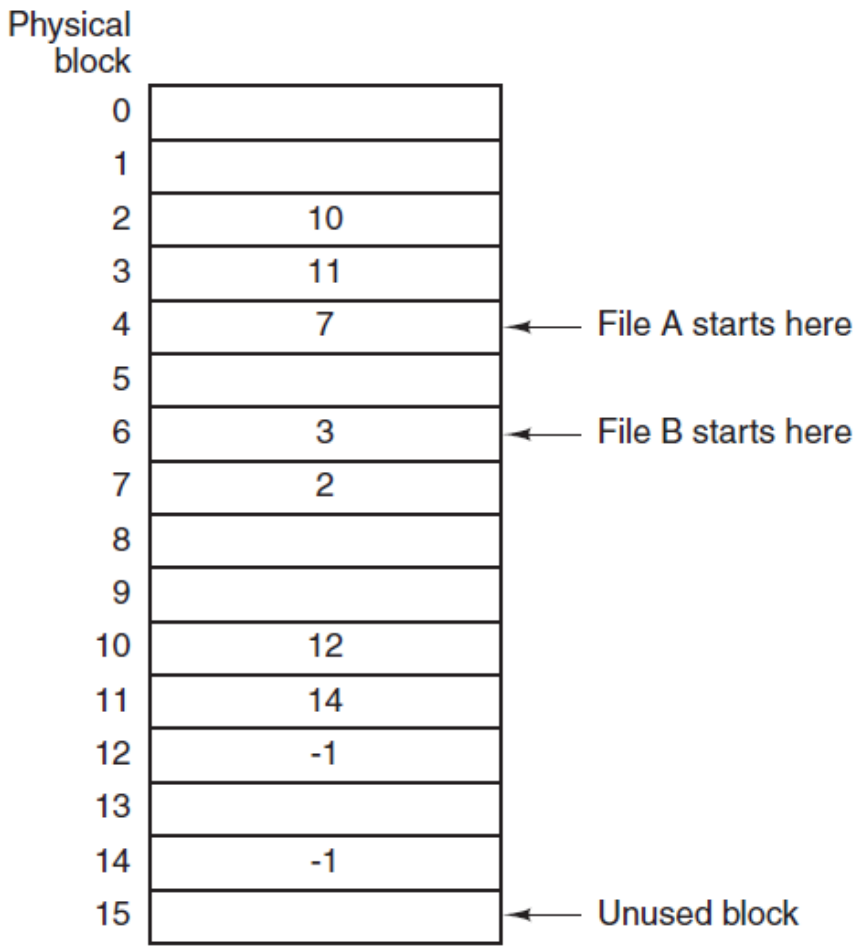
- Unlike contiguous allocation, the data blocks of a file can be stored in any sectors (no need to be next to each other).
- This method creates a table as shown in the figure a side. This table is called a file allocation table (FAT). The FAT is stored in the hard disk.
- Conceptually, we have one FAT.
- The FAT consists of 2 columns.
  - The first column contains the disk/physical block numbers (sector numbers).
  - The second column also contains the sector number that stores the next data block of this file. The -1 number represents the end of a file.
  - The sector who stores the data block 0 will be given in the directory.



Example: File Allocation Table

1) From this FAT, we know that the data blocks 0, 1, 2, 3, and 4 of File A are stored in the sectors 4, 7, 2, 10, and 12, respectively.

2) From this FAT, we know that the data blocks 0, 1, 2, and 3 of File B are stored in the sectors 6, 3, 11, and 14, respectively.



Implementing Files

4) **I-nodes**.

- Unlike contiguous allocation, the data blocks of a file can be stored in any sectors (no need to be next to each other).
- Conceptually, this method creates a table called an index-node (i-node) for each file.
- It list the attributes and sector numbers of the file blocks in order.
- The row below the File Attributes row is referred to the data block 0.
- The address/numbers in the rows specify the sectors (physical blocks) storing the data blocks.
- I-nodes are stored in the hard disk.

Data Block Number	I-Node Table of a File
	File Attributes
0	Physical Block Number
1	Physical Block Number
N-1	Physical Block Number

For example: I-Node tables of Files A and B.

File A I-Node Table

File Attributes
4
7
2
10
12

File B I-Node Table

File Attributes
6
3
11
14

Example: I-Nodes

1) From this i-node table, we know that the data blocks 0, 1, 2, 3, and 4 of File A are stored in the sectors 4, 7, 2, 10, and 12, respectively.

File A
I-Node Table

File Attributes
4
7
2
10
12

2) From this i-node table, we know that the data blocks 0, 1, 2, and 3 of File B are stored in the sectors 6, 3, 11, and 14, respectively.

File B
I-Node Table

File Attributes
6
3
11
14

Example

Consider a computer system storing 100 files.

1) By using Linked-List Allocation, how many file allocation tables (FATs) do we have?

2) By using I-nodes, how many i-node tables do we have?

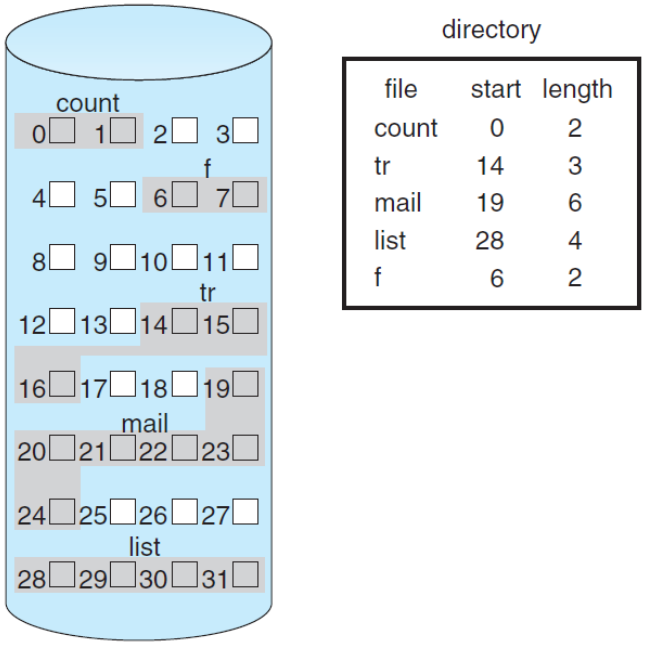
Implementing Directories

Directory is a file containing a list of all files and physical locations (sector numbers) in this directory.

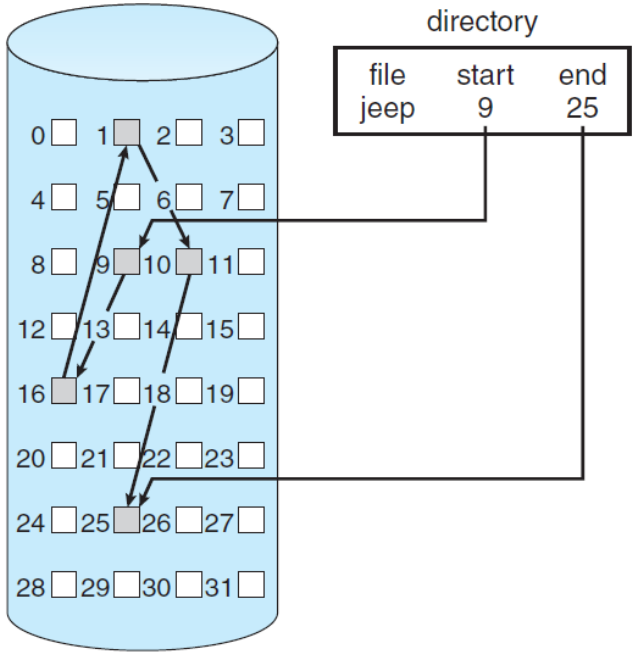
- For the contiguous allocation, the directory will show the file names, their starting sectors (physical blocks), their lengths.
- For the linked-list allocation, the directory will show the file names, their starting sectors, their ending sectors.
- For the i-node allocation, the directory will show the file names, their sectors storing the i-node tables.

Example

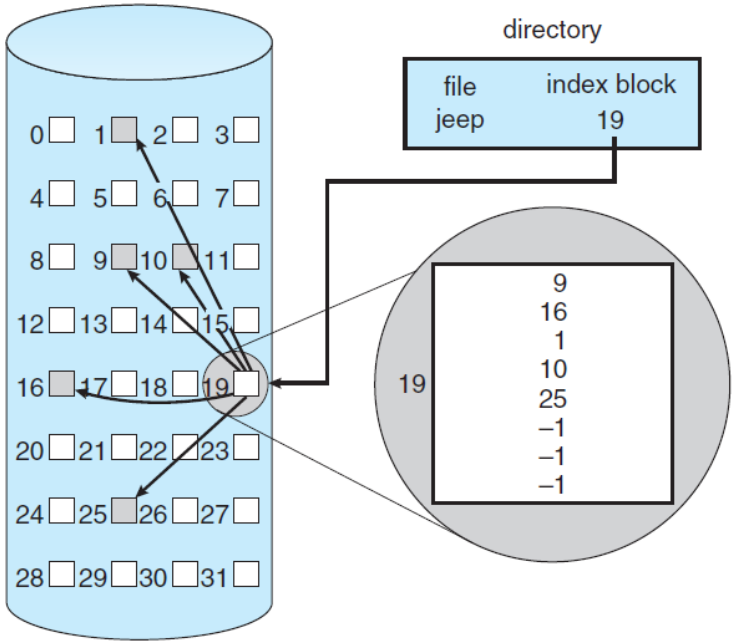
The following figures show the directory and file allocation based on the approaches studied previously.



Contiguous Allocation



Linked-List Allocation



I-Node Allocation

Free-Space Management: Record of Free Sectors

1) Since disk space is limited, we need to reuse the space from deleted files for new files, if possible.
2) To keep track of free disk space, the system maintains a free-space list.
   - The free-space list records all free sectors (physical blocks/disk blocks) – those not allocated to some file or directory.
3) To create a file, we search the free-space list for the required amount of space and allocate that space to the new file. This space is then removed from the free-space list.
4) When a file is deleted, its disk space is added to the free-space list. The free-space list, despite its name, may not be implemented as a list.

Free-Space Management - Bit Vector

1) Frequently, the free-space list is implemented as a bit map or bit vector (i.e., a series of binary numbers). Each sector/physical block is represented by 1 bit.
   - If a sector is free, the bit is 1.
   - If a sector is allocated/occupied (not free), the bit is 0.
2) The main advantage of this approach is its relative simplicity and its efficiency in finding the first free sector or *n* consecutive free sectors on the disk.

Example

- Consider a disk where physical blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the physical blocks are allocated.
- Note that the first physical block is called Block 0.
- As a result, the free-space bit map would be

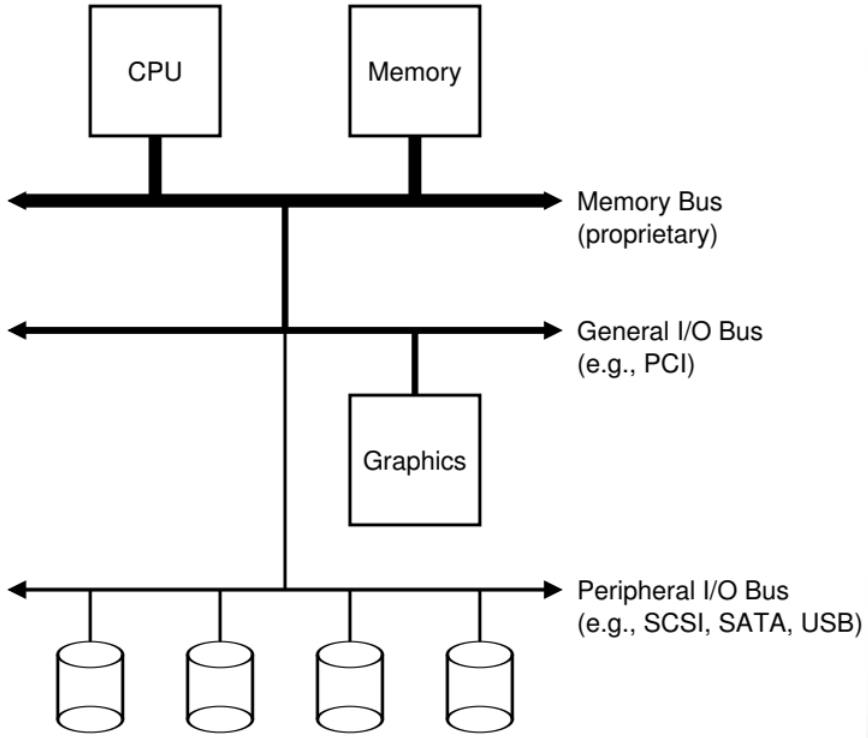
00111100111111000110000011100000 ...

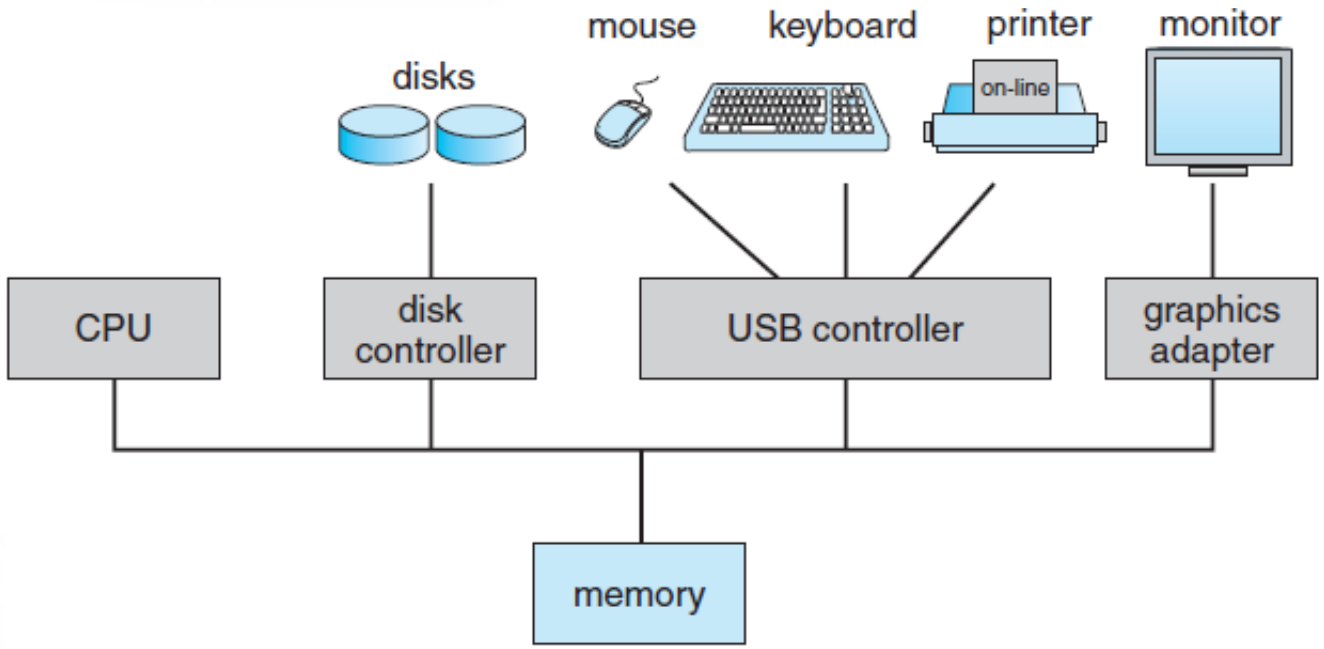
5. Input and Output (I/O) System

1. Computer-System and I/O Hardware

A computer system consists of one or more CPUs and a number of device controllers connected through a common bus that provides access to shared memory.

- Input-output (I/O) devices are connected to this computer via device controllers.
- I/O devices can be classified into storages devices (disk), transmission devices (NIC, Wireless card), human-interface devices (screen, keyboard, mouse), and others.

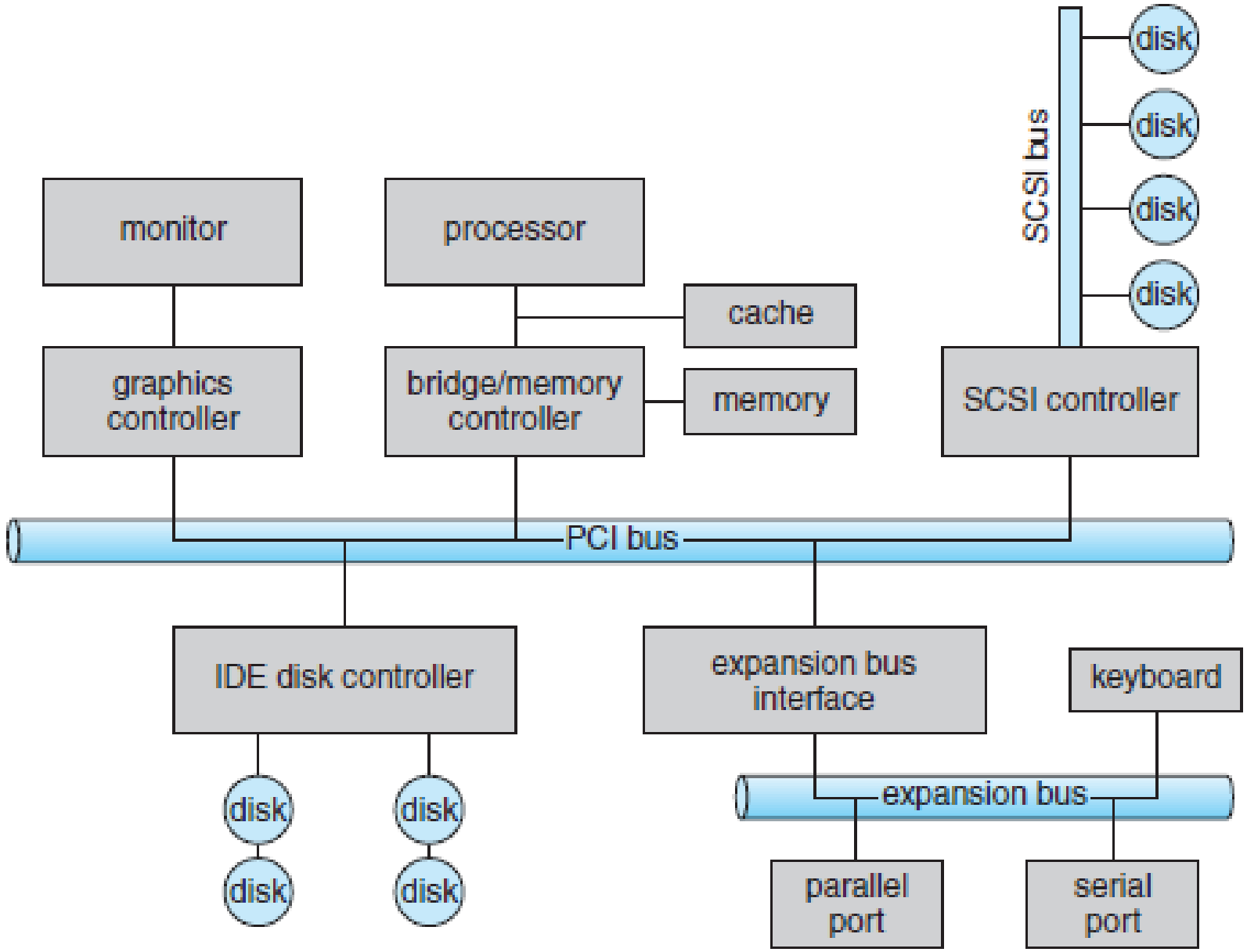




1. Computer-System and I/O Hardware

- I/O Connection
  a) I/O devices can connect to a computer system via a connection point called a *port* (serial ports or parallel ports).
  b) I/O devices can connect to a computer system via a set of wires called a *bus*.
- Three basic types of buses are widely used in computer systems:
  a) The *PCI* (Peripheral Component Interconnect) bus connects high-speed high-bandwidth devices to the memory and CPU subsystems.
  b) An *expansion* bus connects relatively slow devices, such as the keyboard and serial and USB ports.
  c) A *SCSI* (small computer system interface) bus connects a number of SCSI devices to a common SCSI controller.

Example

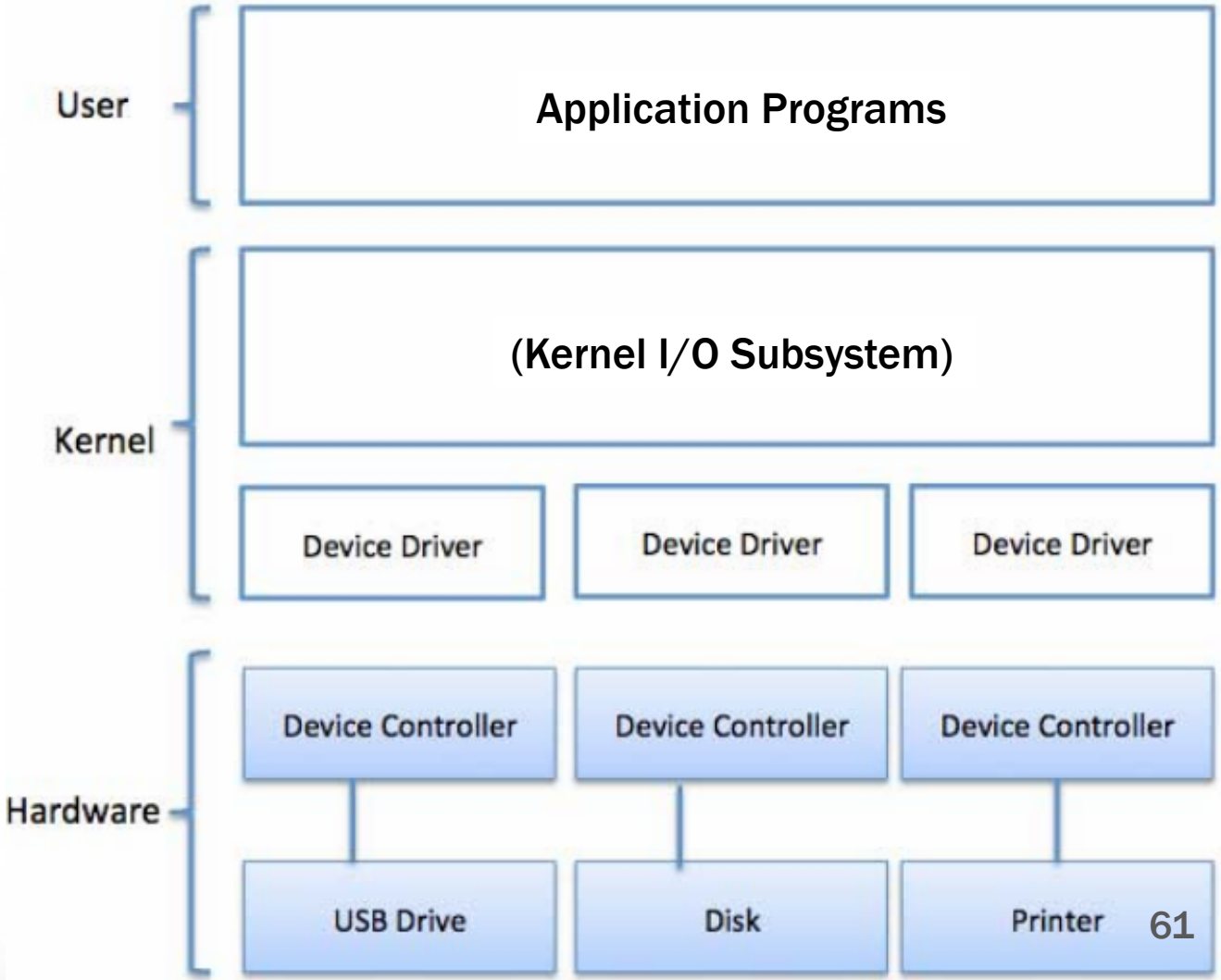

monitor
processor
cache
graphics controller
bridge/memory controller
memory
SCSI bus
disk
disk
disk
disk
SCSI controller
PCI bus
IDE disk controller
expansion bus interface
keyboard
disk
disk
disk
disk
expansion bus
parallel port
serial port

2. Layers of I/O System

App Program ←→ I/O Subsystem (in OS Kernel) ←→ Device Drivers (Add-on S/W interfacing with I/O Subsystem) ←→ Device Controller (H/W) ←→ Bus/Port ←→ Devices

Key components of I/O management:

1) Kernel I/O subsystem. The kernel I/O subsystem is a module in the OS kernel. The kernel I/O subsystem provides the services to the I/O parts, for example, scheduling, buffering, spooling, error handling, protection.
2) Device driver
3) Device controller



2.1 Device Driver

A device driver is an add-on software such that a computer system can interact with a device properly. The computer system can use an I/O device because of its device driver.

- It is written based on using a set of I/O system calls provided by the operating system such as read(), write(), etc.
- It makes the I/O subsystem independent of devices and their controllers.
- A device driver is a part of the operating system working as an interface between application programs and a device.

2.2 Device Controller

- A *device controller* is an electronic component that can operate a port, a bus, or a device. An I/O device connects to a computer system via a device controller.
- A device controller typically consists of four registers:
  a) The *data-in register* is read by the CPU to get input.
  b) The *data-out register* is written by the CPU to send output.
  c) The *status register* contains bits that can be read by the CPU. These bits indicate status, such as:
     i) whether a device is busy or idle (available);
     ii) whether the current command has completed;
     iii) whether a byte is available to be read from the data-in register;
     iv) whether a device error has occurred.
  d) The *control register* can be written by the CPU to start a command or to change the mode of a device, for example, read or write.

3. Device Controller: Basic I/O Operations

- I/O operation refers to basic steps on how a computer system (specifically, CPU) can interact with I/O devices.
- This interaction (sending or receiving data/commands) with I/O devices involves the following hardware units:

CPU ←→ Device Controller ←→ Device

3. Device Controller: I/O Communication Methods

Modern systems primarily use two methods for interaction between the CPU and the Device Controller:

1) Polling-based I/O (Busy-Waiting)
   - The CPU repeatedly checks a "status bit" on the device controller.
   - The CPU is kept in a "busy-wait" loop until the device is ready.
   - Drawback: Inefficient CPU utilization; the CPU cannot perform other tasks while waiting.
2) Interrupt-driven I/O
   - The CPU initiates the I/O request and then moves on to other tasks.
   - When the device is ready, the controller sends an Interrupt Signal to the CPU.
   - The CPU then pauses its current work to execute the Interrupt Service Routine (ISR).
   - Benefit: High efficiency; allows for asynchronous processing.

5. Categories of I/O Devices

1. Data-transfer mode:
   - A character-stream device transfers a byte at a time. (Ex., monitor)
   - A block device transfers a block of bytes at a time. (Ex., disk)

2. Access method (to data, e.x., read, write):
   - A sequential device transfers data in a fixed order. (Ex., tape, modem)
   - A random-access device can transfer data to/from (jump to) wherever area in that device freely. (Ex., CD-ROM, disk)

5. Categories of I/O Devices

3. Data transfer schedule:
   - A synchronous device performs data transfers with predictable response times in coordination with other aspects of the system (we can predict that when an event will happen). (Ex., tape)
   - An asynchronous device exhibits irregular or unpredictable response times not coordinated with other computer events. (Ex., keyboard)

5. Categories of I/O Devices

4. Sharing:
   - A sharable device can be used concurrently by several processes or threads. (Ex., monitor, keyboard)
   - A dedicated device is owned by a process or a thread per time. (Ex., tape)

5. I/O direction:
   - A read-write device has the two-way direction. (Ex., disk)
   - A read-only device has the one-way direction. (Ex., CD-Rom)
   - A write-only device has the one-way direction. (Ex., monitor)

5. Categories of I/O Devices

6. Device speed (ranges from a few bytes per second to a few gigabytes per second as shown below)

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner at 300 dpi	1 MB/sec
Digital camcorder	3.5 MB/sec
4x Blu-ray disc	18 MB/sec
802.11n Wireless	37.5 MB/sec
USB 2.0	60 MB/sec
FireWire 800	100 MB/sec
Gigabit Ethernet	125 MB/sec
SATA 3 disk drive	600 MB/sec
USB 3.0	625 MB/sec

6. Clocks and Timers

1) Most of computer systems have clocks and timers to provide:
   a) the current time,
   b) the elapsed time,
   c) set a timer to trigger an operation at a particular time.
2) A programmable interval timer is the hardware to measure elapsed time and, then, generate an interrupt (sent to other hardware) to trigger operations.
   - For example, the CPU scheduler uses this timer to preempt a process at the end of its time slice.

7. Kernel Buffering

- A buffer is a memory area that temporarily stores data being transferred between two devices or between a device and an application.
- Kernel buffering (the buffer in the kernel space) is done for two reasons.

1. To cope with a speed mismatch between two or more devices who are exchanging data.
   - For example, consider a file being transferred by a modem to a disk. Because the disk is faster than the modem, the data from the modem will stored in a buffers first. Then, the disk will load it later.
2. To provide adaptations for devices that have different data-transfer sizes.
   - For example, a modem might transfer data a byte at a time while the hard disk write the data a block of bytes at a time.

7. Kernel Buffering

In addition to traditional one kernel buffer, to achieve faster data transfer, double buffering means that two kernel buffers are used in data transfer between I/O devices.

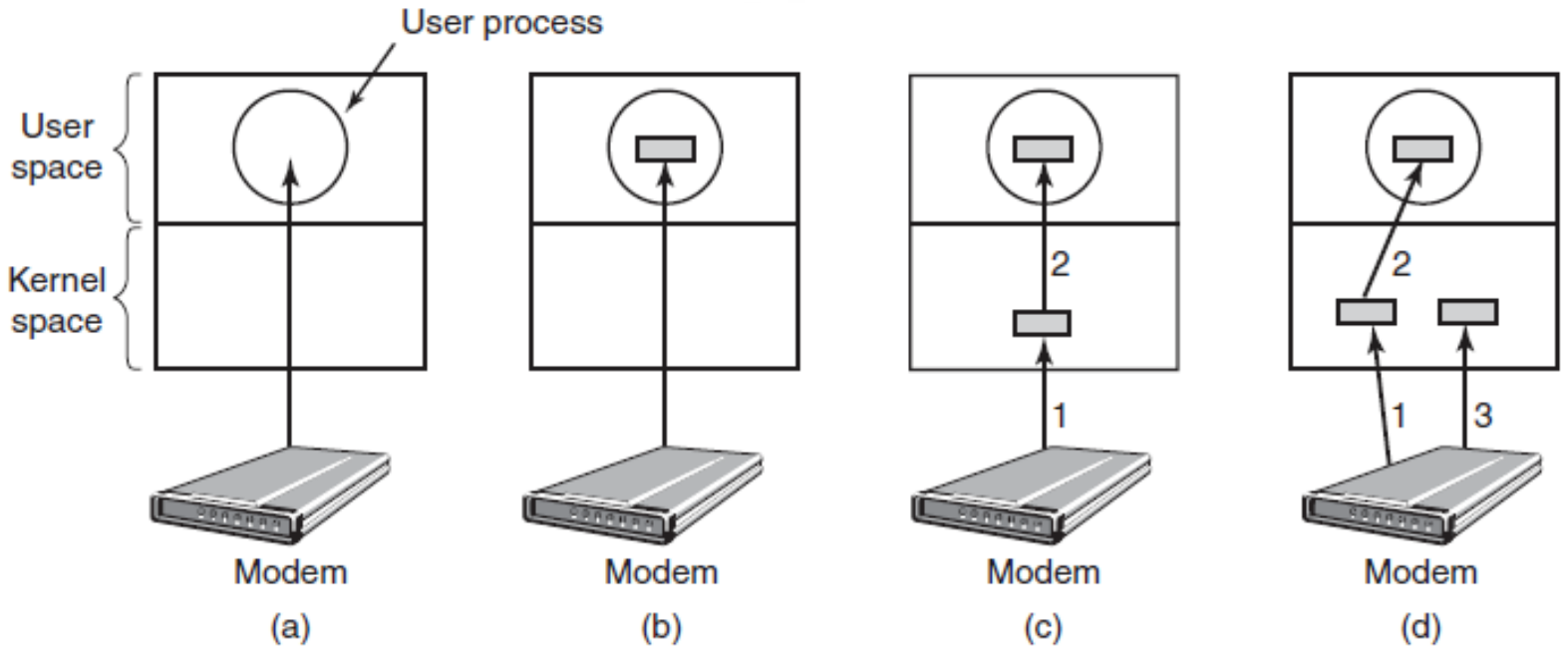


Figure 5-15. (a) Unbuffered input. (b) Buffering in user space. (c) Buffering in the kernel followed by copying to user space. (d) Double buffering in the kernel.

Reading Assignment

- Book “Operating System Concepts”:
  - Section 10.1.1-10.1.4, 10.3.2-10.3.6, 10.6
  - Section 13.1, 13.2-13.2.4, 13.3-13.3.5, 13.4-13.4.6
- Book “Modern Operating System”:
  - 4.3.1, 4.3.2, 4.3.3.

Glossary (by Google NotebookLM)

- File System: A service provided by the operating system for space management in nonvolatile storage devices, enabling efficient storage, location, and retrieval of data.
- Nonvolatile Storage: Storage devices that retain data even when power is removed, such as hard drives.
- File Attributes (Metadata): Information about a file, including its name, type, location, size, date, and protection settings.
- Directory (Folder): A structure used to organize and provide information about files in the system.
- File Operation: Actions that can be performed on a file, such as creating, writing, reading, repositioning, deleting, and truncating.
- Sequential Access: A method of accessing file data in a fixed order, one record after another.
- Direct Access: A method of accessing file data rapidly in any order, based on fixed-length logical records (blocks).
- Partition: A space crafted out of a disk.
- Volume: A partition that has been formatted into a filesystem and contains files and a volume table of contents.

Glossary (by Google NotebookLM)

- Directory Structure: The organizational method used for arranging files into directories, such as single-level, two-level, or tree-structured.
- Path Name: A location of a file in a computer, which can be absolute (starting from the root) or relative (starting from the current directory).
- Protection (File/Directory): Mechanisms used to control who can access files or directories and what actions they can perform (read, write, execute, etc.).
- Contiguous Allocation: An approach to file implementation where data blocks of a file are stored consecutively on the disk.
- Linked-List Allocation: An approach to file implementation where data blocks of a file are linked together by pointers stored within the blocks themselves.
- File Allocation Table (FAT): A table stored in main memory used in Linked-List Allocation using a Table in Memory, which tracks the sequence of disk blocks for each file.
- I-node: A table created for each file in the I-nodes implementation approach, listing attributes and disk addresses of file blocks.

Glossary (by Google NotebookLM)

- Free-Space List: A mechanism used by the system to keep track of available (unallocated) disk blocks.
- Bit Map (Bit Vector): A common implementation for the free-space list where each bit represents a physical block, indicating if it is free (1) or allocated (0).
- I/O (Input/Output): The processes of receiving data from a device (input) or sending data to a device (output).
- Device Controller: An electronic component that operates a port, bus, or device and facilitates communication between the computer system and an I/O device.
- Device Driver: Add-on software that enables a computer system to interact properly with a specific hardware device by translating operating system commands into device-specific instructions.
- Kernel I/O Subsystem: A module within the operating system kernel that provides services for I/O operations, including scheduling, buffering, and error handling.
- Polling-based I/O: An I/O operation where the CPU repeatedly checks (busy-waits) for the device controller and device to be ready.
- Interrupt-driven I/O: An I/O operation where the device controller signals the CPU (sends an interrupt) when it is ready for service.

Glossary (by Google NotebookLM)

- Character-stream device: An I/O device that transfers data one byte at a time.
- Block device: An I/O device that transfers data in blocks of bytes.
- Buffer: A memory area used to temporarily store data being transferred between devices or between a device and an application.
- Double Buffering: Using two buffers in a data transfer process to improve efficiency.